АЙСМАРТ

Как ИИ использует данные

Занятие 4. Как ИИ использует данные

Содержание

- 1. Вступление-обращение
- 2. Теоретический материал
- 2.1. Что такое искусственный интеллект
- 2.2. Данные — это опыт для модели
- 2.3. Четыре распространённых типа задач ИИ
- 2.4. Жизненный цикл ИИ-решения
- 2.5. Сквозной пример: рекомендации книг в школьной библиотеке
- 2.6. Почему ИИ ошибается
- 3. Дополнительные задания
- Дополнительная задача 1. Критика набора данных
- Дополнительная задача 2. Цена ошибки
- 4. Итоги урока
- Фраза для завершения
- 5. Мини-глоссарий урока
- 6. Рекомендация по среде
- Материалы к занятию

Цель урока: сформировать понятное и целостное представление о том, зачем ИИ-системам нужны данные, какие задачи они решают и как устроен простой жизненный цикл ИИ-решения — от постановки задачи до проверки результата.

Особое внимание уделяется связи между этапами: качество ответа модели зависит не только от выбранного алгоритма, но и от того, насколько точно сформулирована задача, подходят ли данные и корректно ли организована проверка.

1. Вступление-обращение

Привет!

Ты уже знаешь, что программа получает данные, выполняет действия и выдаёт результат.

Сегодня мы сделаем следующий шаг и разберёмся, как на этой идее строятся системы искусственного интеллекта. Ты увидишь, почему для ИИ данные похожи на опыт человека, чем распознавание отличается от прогноза и зачем результат модели обязательно проверять на новых примерах. К концу урока ты сможешь разложить почти любой знакомый ИИ-сервис на четыре понятных шага: задача, данные, модель и проверка.

Во время урока обращай внимание не только на то, что умеет система, но и на условия её работы. Один и тот же ИИ-сервис может давать хорошие ответы на знакомых данных и ошибаться, когда получает необычный или ранее не встречавшийся пример.

2. Теоретический материал

2.1. Что такое искусственный интеллект

Искусственный интеллект (ИИ) — это область информатики, в которой создают системы, способные выполнять задачи, обычно требующие от человека восприятия, выбора, поиска закономерностей или принятия решения. К таким задачам относятся распознавание речи, определение объектов на фотографии, прогноз спроса, подбор музыки и многие другие.

Важно понимать: ИИ — это не одна конкретная программа и не обязательно робот. Чаще всего ИИ скрыт внутри обычного цифрового сервиса: камеры телефона, поисковой системы, навигатора, банковского приложения или образовательной платформы.

Внешне работа такого сервиса может выглядеть как одно действие: пользователь загружает фотографию или вводит текст и сразу получает ответ. Однако внутри система должна представить вход в форме данных, обработать его с помощью модели и преобразовать результат в понятный пользователю вид.

Бытовая аналогия

Обычная программа по заранее заданным правилам похожа на ученика, которому дали точную инструкцию: «если произошло А — делай Б». Модель машинного обучения больше похожа на ученика, которому показали много примеров и предложили самостоятельно заметить закономерность.

Аналогия не означает, что модель думает так же, как человек. Она подчёркивает только различие между явно записанными правилами и настройкой по примерам.

Программа по правилам и модель по данным

Признак	Программа по правилам	Модель, обученная на данных
Откуда берётся способ решения	Правила записывает программист	Закономерности настраиваются по примерам
Подходит для	Чётких и полностью описываемых условий	Сложных случаев, где трудно перечислить все правила

Пример	Если пароль короче 8 символов — показать предупреждение	Определить, является ли письмо спамом
Главный риск	Программист не учёл правило	Данные были плохими или не отражали реальную ситуацию

Это различие не означает, что один подход всегда лучше другого. Если условие известно точно, обычное правило часто проще проверить и объяснить. В реальных сервисах правила и обученные модели могут работать вместе: модель определяет категорию сообщения, а программа по заранее заданному правилу выбирает дальнейшее действие.

Пример 1. Решение по заранее записанному правилу

```
temperature = -5

if temperature < 0:
 message = "На улице мороз"
else:
 message = "Температура выше нуля"

print(message)
```

Этот код не обучается на данных: программист сам записал границу 0 °C и оба возможных действия. Такая программа полезна, когда правило известно заранее и его легко сформулировать.

При значении переменной temperature, равном −5, условие temperature < 0 истинно, поэтому в message записывается строка «На улице мороз». Если temperature будет равно 0, выполнится ветвь else, так как число 0 не меньше нуля.

В задачах ИИ ситуация обычно сложнее. Например, невозможно надёжно описать словами все признаки фотографии кошки: разные позы, освещение, фон, размер и качество изображения создают слишком много вариантов. Поэтому системе показывают множество примеров и позволяют модели настроиться на них.

При этом модель не получает универсального правила «что такое кошка». Она настраивается на закономерности, представленные в учебных примерах, поэтому необычная фотография может привести к ошибке.

Проверь понимание Почему программа с условием if ещё не обязательно является ИИ? Назови цифровой сервис, в котором ИИ может быть скрыт внутри обычного интерфейса. В каком случае заранее записанное правило будет уместнее обучаемой модели?

2.2. Данные — это опыт для модели

Данные — это зафиксированные сведения, которые можно хранить, передавать и обрабатывать.

Для человека опытом могут быть наблюдения, тренировки и обратная связь. Для модели роль опыта выполняют данные: примеры объектов и правильных результатов, на которых она учится замечать повторяющиеся связи.

Данными для ИИ могут быть числа, тексты, фотографии, звук, видео, координаты, действия пользователя, показания датчиков или комбинация нескольких видов информации. Вид данных зависит от задачи.

Одни и те же сведения могут играть разную роль в разных задачах. Например, цена товара может быть признаком при прогнозировании спроса и правильным ответом, то есть меткой, при задаче прогнозирования самой цены.

Понятие	Простое объяснение	Пример
Объект	То, о чём модель получает информацию	одно письмо, одна фотография, один товар
Признак	Свойство объекта, которое помогает принять решение	длина текста, цвет пикселей, цена товара
Метка	Правильный ответ для учебного примера	«спам», «кошка», фактическая цена
Набор данных	Много объектов, собранных вместе	таблица писем с признаками и метками

Не каждый набор данных обязательно содержит метки. Например, можно собрать тексты без заранее указанных категорий. Однако в рассматриваемых далее учебных примерах классификации метки нужны, чтобы сравнивать ответы модели с правильными категориями.

Пример 2. Как учебные примеры можно представить в Python

```
emails = [
{"text": "Получите приз прямо сейчас", "label": "спам"},
{"text": "Встречаемся после уроков", "label": "не спам"},
{"text": "Скидка только сегодня", "label": "спам"}
]

for email in emails:
 print(email["text"], "⊠", email["label"])
```

Каждый словарь в списке описывает один объект — письмо. Поле text содержит исходные данные, а поле label — правильную категорию. Реальный набор данных намного больше, а тексты и метки проходят проверку.

Цикл for по очереди обращается к каждому письму. Выражения email["text"] и email["label"] извлекают значения из соответствующих полей, поэтому рядом выводятся текст объекта и связанная с ним метка.

Важно не считать отдельные слова готовыми и безошибочными правилами. Например, слово «скидка» может встретиться и в обычной переписке. Модель должна учитывать закономерности в наборе примеров, а качество этих закономерностей затем проверяется на новых письмах.

Какими должны быть данные

Релевантными.

Данные должны относиться именно к решаемой задаче. Для распознавания дорожных знаков мало фотографий комнатных растений.

Точными.

Ошибочные подписи и неверные измерения учат модель неправильным связям.

Достаточными.

Несколько примеров редко отражают всё разнообразие реального мира. Достаточность определяется не одним фиксированным числом, а сложностью задачи и разнообразием возможных ситуаций.

Разнообразными.

Нужно учитывать разные условия: освещение, устройства, возраст объектов, формулировки текста и другие варианты.

Сбалансированными.

Если одна категория встречается почти всегда, модель может научиться выбирать её автоматически и при этом плохо решать задачу. Например, при 95 письмах «не спам» и 5 письмах «спам» постоянный ответ «не спам» даст высокую общую долю совпадений, но не поможет находить спам.

Допустимыми к использованию.

Сбор данных должен учитывать приватность, согласие, авторские права и правила безопасности. Техническая возможность получить сведения ещё не означает, что их разрешено и уместно использовать.

Почему «больше данных» не всегда означает «лучше»

Тысяча неверно размеченных примеров может быть хуже ста качественных. Важны не только объём, но и точность, разнообразие и соответствие задаче.

Кроме того, многократное повторение почти одинаковых объектов увеличивает размер набора, но почти не добавляет модели нового опыта.

Смещение данных

Смещение данных возникает, когда набор примеров представляет реальный мир односторонне. Допустим, систему распознавания школьных рюкзаков обучили только на фотографиях одного цвета и одного ракурса. На новых изображениях она может ошибаться не потому, что «не

старается», а потому что нужного опыта в данных не было.

Смещение может появиться ещё на этапе сбора: например, если фотографии делают только одним устройством или только в одном помещении. Оно также может возникнуть при разметке, если сложные случаи исключают из набора или разные специалисты понимают категории неодинаково.

Поэтому разработчики не только собирают данные, но и анализируют, какие случаи представлены, какие отсутствуют и для каких групп пользователей качество может различаться.

Проверь понимание Что в наборе писем является объектом, признаком и меткой? Почему набор из очень похожих фотографий может плохо подготовить модель к реальной работе? Какую проблему создаёт неверная разметка данных? Можно ли считать набор сбалансированным только потому, что в нём много объектов?

2.3. Четыре распространённых типа задач ИИ

Один и тот же общий подход — использовать данные и модель — применяется к разным задачам. Чтобы понимать, какой результат ожидается от системы, полезно различать четыре распространённых типа.

Тип задачи удобнее определять не по названию сервиса, а по его выходу. Категория указывает на классификацию, будущее число — на прогнозирование, а упорядоченный набор вариантов — на рекомендации.

Распознавание

Вход:

изображение, звук

Выход:

найденный объект или текст

Пример: текст на фотографии

Классификация

Вход:

объект и его признаки

Выход:

категория (метка)

Пример: «спам» или «не спам»

Прогнозирование

Вход:

история значений и условия

Выход:

число или вероятность

Пример: время прибытия автобуса

Рекомендации

Вход:

пользователь, объекты, контекст

Выход:

упорядоченный список

Пример: подборка книг

Рисунок 1. Четыре распространённых типа задач ИИ

Распознавание

Распознавание отвечает на вопрос «что обнаружено в сигнале или изображении?». Система может находить буквы в скане, слова в аудиозаписи, лица или дорожные знаки на изображении. Часто распознавание включает несколько действий: сначала найти объект, затем определить, что это за объект.

Например, при распознавании текста на фотографии система должна обнаружить области с символами и представить их как последовательность букв. Ошибка может возникнуть как при поиске области, так и при чтении отдельного символа.

Классификация

Классификация относит объект к одной из заранее определённых категорий. Входом может быть письмо, фотография или набор измерений, а выходом — метка: «спам», «не спам», «кошка», «собака», «высокий риск», «низкий риск».

Категории должны быть определены до проверки результата. Если модель должна выбрать между «спамом» и «не спамом», её ответ нельзя оценивать по категории, которой не было в постановке задачи.

Не путай

Распознавание часто связано с обнаружением и чтением содержимого, а классификация — с выбором категории. В реальной системе они могут идти подряд. Например: сначала камера распознаёт дорожный знак, затем классифицирует его как «ограничение скорости».

Название конкретной функции может использоваться по-разному, поэтому при анализе системы всегда уточняй, что именно поступает на вход и какой результат появляется на выходе.

Прогнозирование

Прогнозирование оценивает будущее значение или вероятность события на основе прошлых наблюдений. Примеры: прогноз спроса на товар, времени прибытия автобуса, нагрузки на сервер или вероятности поломки оборудования. Прогноз не является гарантией: он показывает наиболее вероятный результат при имеющихся данных.

Проверить такой ответ сразу удаётся не всегда: нужно дождаться фактического события или значения. Например, прогноз времени прибытия сравнивают с реальным временем после того, как транспорт прибыл.

Рекомендации

Рекомендательная система выбирает и упорядочивает варианты, которые могут быть полезны конкретному пользователю. Она может учитывать историю просмотров, оценки, содержание объектов, действия похожих пользователей и контекст. Результат обычно представляет собой не одну категорию, а список, отсортированный по предполагаемой полезности.

У рекомендации может не быть единственного правильного ответа. Две разные подборки способны оказаться полезными, поэтому качество оценивают по действиям пользователя, обратной связи и свойствам списка, например его разнообразию.

Пример 3. Упрощённая иллюстрация оценки вариантов

```
books = [
{"title": "Космос для начинающих", "topic": "наука"},
{"title": "Тайна старого дома", "topic": "детектив"},
{"title": "Эксперименты дома", "topic": "наука"}
]

favorite_topic = "наука"

for book in books:
 score = 1 if book["topic"] == favorite_topic else 0
 print(book["title"], "— балл", score)
```

Этот пример ещё не является полноценной ИИ-моделью: правило подсчёта балла записано вручную. Но он показывает важную идею рекомендаций — каждому варианту присваивается оценка, после чего варианты можно упорядочить. Реальная модель учится рассчитывать такие оценки по большому числу примеров и признаков.

В данном списке книги по теме «наука» получают балл 1, а книга по теме «детектив» — балл 0. Если несколько книг получили одинаковый балл, этого единственного признака недостаточно, чтобы определить их порядок между собой.

Тип задачи	Типичный вход	Типичный выход	Как проверить

Распознавание	изображение, звук	найденный объект или текст	сравнить с тем, что действительно было в сигнале
Классификация	объект и его признаки	категория	сравнить с правильной меткой
Прогнозирование	история значений и условия	число или вероятность	сравнить прогноз с фактом
Рекомендации	пользователь, объекты, контекст	упорядоченный список	оценить полезность, выбор и обратную связь
Проверь понимание К какому типу относится задача «определить, является ли отзыв положительным»? Чем прогноз времени прибытия отличается от классификации письма? Почему рекомендация обычно представляет собой список, а не один правильный ответ? Какой тип задачи выдаёт категорию, а какой — значение, относящееся к будущему?			

2.4. Жизненный цикл ИИ-решения

ИИ-решение не начинается с выбора нейросети или написания кода. Сначала нужно понять задачу. После этого подбирают данные, создают или обучают модель и обязательно проверяют результат. Если качество недостаточно, цикл повторяется.

Каждый этап влияет на остальные. Изменение ожидаемого выхода может потребовать новых меток, а обнаруженная при проверке ошибка может показать, что в данных отсутствует важная группа примеров.

Связь со схемой из занятия 2

В занятии 2 мы разбирали подробную цепочку из шести этапов: постановка задачи ☒ входные данные ☒ алгоритм обработки ☒ модель ☒ результат ☒ проверка качества. Здесь те же этапы собраны в четыре шага: алгоритм и модель объединены в шаге «Обучить модель», а получение результата и проверка качества — в шаге «Проверить результат».

1. Задача

вход и выход

2. Данные

сбор и разметка

3. Модель

учится на примерах

4. Проверка

метрики, ошибки

Если качество недостаточно — цикл повторяется

Рисунок 2. Упрощённый жизненный цикл ИИ-решения

Шаг 1. Сформулировать задачу

Хорошая постановка задачи отвечает как минимум на четыре вопроса:

- Кто будет использовать результат?
- Что поступает на вход системы?
- Что система должна выдать на выходе?
- По какому признаку мы поймём, что результат достаточно хороший?

Формулировка «сделать умный сервис» слишком расплывчата. Формулировка «по тексту входящего сообщения определить, относится ли оно к вопросу об оплате, доставке или возврате» уже задаёт понятный вход, выход и категории.

На этом этапе также важно определить ограничения. Например, что делать с сообщением, которое относится сразу к двум категориям, написано на неподдерживаемом языке или не относится ни к одной из заданных тем.

Шаг 2. Подготовить данные

На этом этапе определяют, какие примеры нужны, откуда их можно получить и как проверить качество. Для задач с правильными ответами данные часто размечают: каждому объекту назначают метку. Затем набор делят как минимум на учебную и тестовую части.

Разделение выполняют до итоговой проверки. Один и тот же объект не должен одновременно помогать модели настраиваться и изображать новый пример, иначе оценка может оказаться завышенной.

Аналогия с контрольной работой

Учебные данные — это задачи, по которым ученик готовится. Тестовые данные — новая контрольная, которую он раньше не видел. Если проверять только на знакомых примерах, нельзя понять, научилась ли модель решать задачу или просто «запомнила ответы».

Поэтому тестовые примеры не используют для подгонки ответа модели во время обучения.

Шаг 3. Обучить модель

Модель — это математически настроенный способ преобразовать входные данные в результат.

Во время обучения алгоритм изменяет внутренние параметры модели так, чтобы её ответы на учебных примерах становились ближе к правильным. Внутренние параметры можно представить как множество числовых настроек, которые модель подбирает автоматически.

На этом уроке не требуется разбирать формулы обучения. Важно понять общий принцип: модель получает примеры, делает предположения, сравнивает их с правильными ответами и постепенно корректирует свои настройки.

Обучение и использование модели — разные действия. При обучении параметры изменяются по примерам, а при обычном использовании уже настроенная модель получает новый объект и выдаёт предсказание.

Шаг 4. Проверить результат

После обучения модель проверяют на новых данных. Разработчики считают показатели качества, рассматривают ошибки и выясняют, в каких ситуациях модель работает хуже. Для классификации простым показателем может быть доля правильных ответов — accuracy.

Пример 4. Подсчёт доли правильных ответов

```
predictions = ["кошка", "собака", "кошка", "собака"]
correct_answers = ["кошка", "кошка", "кошка", "собака"]

correct = 0

for prediction, answer in zip(predictions, correct_answers):
 if prediction == answer:
 correct += 1

accuracy = correct / len(correct_answers)
print(accuracy)
```

Модель дала три правильных ответа из четырёх, поэтому программа выведет 0.75. Это означает 75% правильных ответов на данном наборе. Одного показателя не всегда достаточно: важно также посмотреть, какие именно ошибки совершены и насколько они опасны для задачи.

Функция zip объединяет элементы двух списков попарно: первое предсказание сравнивается с первым правильным ответом, второе — со вторым и так далее. При совпадении счётчик correct увеличивается на единицу. Затем число совпадений делится на общее количество правильных ответов.

В таком подсчёте предполагается, что списки соответствуют одним и тем же объектам и ответы расположены в одинаковом порядке. Если порядок нарушен, сравнение потеряет смысл.

Качество зависит от контекста

75% может быть неплохим результатом для чернового учебного прототипа и недопустимым для

системы, связанной с безопасностью или важными решениями. Требования к качеству задают до разработки, исходя из последствий ошибки.

Даже высокое среднее значение не гарантирует одинаковую работу во всех условиях, поэтому результаты дополнительно рассматривают по категориям и группам примеров.

Цикл повторяется

Если проверка показывает слабое качество, разработчики возвращаются назад: уточняют задачу, собирают более подходящие данные, исправляют разметку, выбирают другие признаки или модель. Поэтому жизненный цикл ИИ правильнее представлять не прямой линией, а повторяющимся циклом улучшений.

Повторная проверка нужна после каждого значимого изменения. Улучшение результата на одной группе примеров может сопровождаться ухудшением на другой, поэтому новое решение снова оценивают целиком.

Проверь понимание Почему нельзя начинать проект с фразы «выберем самую современную модель»? Зачем отделять тестовые данные от учебных? Что нужно делать после получения числовой метрики качества? Чем обучение модели отличается от получения предсказания?

2.5. Сквозной пример: рекомендации книг в школьной библиотеке

Рассмотрим упрощённый проект, который рекомендует ученику книги. Этот пример показывает, как четыре этапа связываются между собой.

До выбора модели необходимо уточнить, что рекомендация должна быть не только интересной, но и выполнимой: отсутствующую в библиотеке книгу нельзя выдать, даже если модель оценила её высоко.

Этап	Решение в проекте	Вопросы для проверки
1. Задача	Предложить ученику пять книг, которые могут заинтересовать его и доступны в библиотеке.	Что считать полезной рекомендацией? Нужно ли учитывать возрастные ограничения и наличие книги?
2. Данные	Темы и описания книг, возрастная категория, история выдач, оценки или отметки «понравилось».	Есть ли данные о разных жанрах? Можно ли использовать историю чтения с учётом приватности?
3. Модель	Оценить соответствие книги интересам ученика и упорядочить подходящие варианты.	Не рекомендует ли система только самые популярные книги?

4. Проверка	Проверить, выбирают ли ученики рекомендации, дочитывают ли книги, довольны ли подборкой.	Как отделить случайный выбор от действительно полезной рекомендации?

Этот проект нельзя оценить только вопросом «угадала ли модель одну правильную книгу», потому что у пользователя может быть несколько подходящих вариантов. Поэтому для рекомендаций используют сочетание показателей: клики, выбор, оценку полезности, разнообразие списка и обратную связь. Кроме того, обязательно применяются правила безопасности: возрастные ограничения, доступность книги и защита персональных данных.

Например, система, которая всегда предлагает только самые популярные книги, может получать много выборов, но почти не учитывать индивидуальные интересы. Анализ качества должен показать не только общую популярность рекомендаций, но и то, помогают ли они находить подходящие варианты разным пользователям.

2.6. Почему ИИ ошибается

Ошибки модели — ожидаемая часть работы с ИИ. Их нужно не скрывать, а измерять и разбирать. Ошибка не всегда указывает на одну конкретную причину: иногда одновременно влияют постановка задачи, данные и условия использования.

Наиболее распространённые причины:

Недостаточно примеров.

Модель не увидела нужного разнообразия ситуаций.

Неверные или противоречивые метки.

Одинаковые объекты подписаны по-разному, поэтому модель получает путаный сигнал.

Данные не похожи на реальную среду.

Учебные фотографии сделаны в студии, а система работает на улице.

Изменилась ситуация.

Поведение пользователей, язык сообщений или ассортимент товаров со временем меняются. Поэтому качество, измеренное перед запуском, не гарантирует такого же результата спустя длительное время.

Неправильно поставлена задача.

Модель оптимизирует показатель, который не отражает реальную полезность.

Слишком большая уверенность в автоматическом ответе.

Даже хорошая модель может ошибиться в редком или необычном случае. Числовая уверенность модели также не превращает ответ в доказанный факт.

Правило ответственного использования

Результат ИИ — это вычисленный ответ, а не абсолютная истина. Чем выше цена ошибки, тем

важнее независимая проверка, понятные ограничения системы и участие человека.

Пользователь должен понимать, для каких задач система предназначена и в каких случаях её ответ нельзя применять без дополнительной проверки.

Как улучшать результат

- уточнить, какую проблему действительно нужно решить;
- собрать более точные и разнообразные данные;
- проверить разметку и удалить явные ошибки;
- разделить качество по разным группам и ситуациям, а не смотреть только на среднее число;
- анализировать примеры, на которых модель ошиблась;
- повторно тестировать систему после изменений и после запуска.

Анализ ошибок должен приводить к конкретному действию. Если модель плохо распознаёт изображения при слабом освещении, следует отдельно проверить наличие таких примеров в данных, их разметку и качество работы именно в этом условии.

3. Дополнительные задания

Дополнительная задача 1. Критика набора данных

Условие

Система должна распознавать школьные доски. В наборе данных есть 500 фотографий одной и той же зелёной доски, снятых в одном кабинете. Объясни минимум две причины, по которым модель может плохо работать в других школах, и предложи, какие примеры добавить.

При ответе рассмотри разнообразие цвета, формы, освещения, ракурса, фона и устройства съёмки. Недостаточно предложить только увеличить количество фотографий: новые примеры должны добавлять условия, которых не было в исходном наборе.

Дополнительная задача 2. Цена ошибки

Условие

Сравни две системы: рекомендации музыки и обнаружение препятствия беспилотным устройством. Почему одинаковая доля правильных ответов может оцениваться по-разному? Какие дополнительные проверки нужны во втором случае?

Укажи возможные последствия ошибочного ответа и объясни, почему для системы обнаружения препятствий важно отдельно проверять редкие, сложные и опасные ситуации, а не ограничиваться одной средней метрикой.

4. Итоги урока

Главная мысль

ИИ использует данные как источник опыта: по примерам модель настраивает закономерности, а затем применяет их к новым объектам. Но качество результата зависит от постановки задачи, данных и проверки — не только от выбранного алгоритма.

Поэтому оценивать ИИ-систему нужно целиком: от смысла задачи и допустимости данных до последствий ошибочного ответа.

- ИИ — это системы, которые решают задачи восприятия, классификации, прогноза, выбора и другие задачи, обычно требующие интеллектуальной обработки информации.
- Данные состоят из объектов и их признаков; в обучающих задачах к ним могут добавляться правильные ответы — метки.
- Качество данных определяется не только количеством, но и точностью, разнообразием, сбалансированностью и допустимостью использования.
- Распознавание находит или читает содержимое, классификация выбирает категорию, прогнозирование оценивает будущее, рекомендации упорядочивают подходящие варианты.
- Жизненный цикл ИИ-решения: задача ☒ данные ☒ модель ☒ проверка результата. При необходимости цикл повторяется.
- Ошибки ИИ нужно измерять, анализировать и учитывать при принятии решений.
- Чем серьёзнее последствия ошибки, тем строже должны быть проверка, ограничения автоматизации и контроль со стороны человека.

Фраза для завершения

Закончи предложение одним–двумя предложениями: «Чтобы создать полезное ИИ-решение, недостаточно просто выбрать модель, потому что…»

В ответе постарайся связать не менее двух элементов урока: постановку задачи, качество данных, проверку на новых примерах или последствия ошибок.

5. Мини-глоссарий урока

Термин	Простое объяснение
Искусственный интеллект	Область создания систем, выполняющих задачи, связанные с восприятием, поиском закономерностей и выбором результата.
Данные	Зафиксированные сведения, которые можно хранить и обрабатывать.
Объект	Один пример, о котором система получает информацию.

Признак	Свойство объекта, используемое для решения задачи.
Метка	Правильный ответ, связанный с учебным примером.
Модель	Настроенный по данным способ преобразовать вход в предсказание.
Обучение	Процесс настройки параметров модели по примерам.
Предсказание	Ответ модели для конкретного входного объекта.
Тестовые данные	Новые примеры, не использованные для обучения и нужные для проверки.
Метрика	Числовой показатель качества результата.
Accuracy	Доля правильных ответов среди всех проверенных примеров.
Смещение данных	Одностороннее представление реальности в наборе данных.
Рекомендательная система	Система, которая оценивает и упорядочивает подходящие варианты для пользователя.

Термины связаны между собой: объект описывается признаками, модель преобразует эти данные в предсказание, а метка позволяет проверить ответ и вычислить метрику качества.

6. Рекомендация по среде

Чтобы самостоятельно проверить примеры кода, можно использовать установленный Python в IDLE или Visual Studio Code. Запуск внешней среды не является обязательной частью теоретического занятия: код можно разобрать по шагам на экране преподавателя. Не вводите в учебные примеры реальные персональные данные.

При самостоятельном запуске важно сохранить отступы внутри циклов и условий: в Python они показывают, какие команды относятся к соответствующему блоку. Для эксперимента можно менять только исходные значения и заранее предполагать результат, а затем сравнивать предположение с выводом программы.

Материалы к занятию

- presentation_Как ИИ использует данные.pptx (1.0 МБ)